

Ex.No.8 Use Steg-Expose to detect hidden data in images

Description

StegExpose is a tool used to detect hidden data in images using steganography analysis. It works by evaluating the statistical properties of an image to estimate whether any hidden data is embedded. Below is a step-by-step guide on how to use StegExpose to detect hidden data in images:

Prerequisites:

1. **Java Runtime Environment (JRE):** StegExpose is a Java-based tool, so ensure you have JRE installed.
2. **StegExpose Tool:** Download StegExpose from the [official GitHub repository](#).

Step-by-Step Process:

1. Download and Set Up StegExpose:

- **Download the tool:** Go to [StegExpose GitHub](#) and download the .jar file.
- **Install Java:** Ensure that Java is installed on your machine. You can download it from the official site.
- **Prepare environment:** Place the .jar file in a folder where you plan to work on the image files.

2. Select Images for Analysis:

- Collect the images you suspect might have hidden data. StegExpose supports formats such as .png, .jpg, .bmp, etc.

3. Open Command Line or Terminal:

- Navigate to the folder where you have saved the StegExpose.jar file.
- Open your **Command Prompt** (Windows) or **Terminal** (Linux/macOS).

4. Run StegExpose on an Image:

- Use the following command to check a single image for hidden data:

```
bash
```

```
java -jar StegExpose.jar <image_file_path>
```

Example:

```
bash
```

```
java -jar StegExpose.jar test_image.png
```

- Replace <image_file_path> with the actual path of the image you want to check.

5. Analyze the Output:

- After running the tool, it will analyze the image and provide a score.
- **Output Explanation:**
 - StegExpose calculates a "suspect" score (between 0 and 1). The higher the score, the more likely that hidden data is present.
 - **Threshold values** (suggested by the tool):
 - **Less than 0.2:** Image is clean (no hidden data).

- **0.2 - 0.3:** Possibly some hidden data.
- **Above 0.3:** Likely that steganography is present.

6. Batch Analysis (Multiple Images):

- If you have multiple images to check, you can analyze them in bulk by specifying a folder. For example:

bash

```
java -jar StegExpose.jar <folder_path>
```

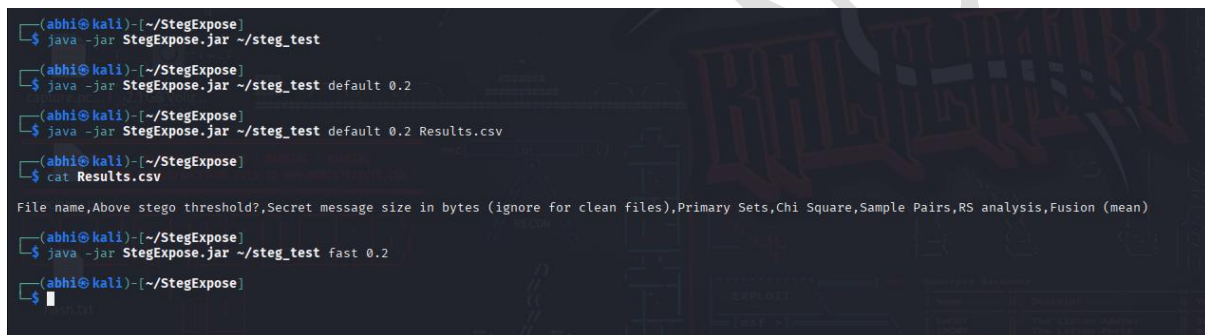
- Replace <folder_path> with the path to the folder containing the images.

7. Advanced Options (Optional):

- StegExpose provides additional parameters, such as adjusting the sensitivity of detection and output verbosity. Use --help to see the list of options:

bash

```
java -jar StegExpose.jar --help
```



```
(abhi@kali)~[/StegExpose]
$ java -jar StegExpose.jar ~/steg_test
(abhi@kali)~[/StegExpose]
$ java -jar StegExpose.jar ~/steg_test default 0.2
(abhi@kali)~[/StegExpose]
$ java -jar StegExpose.jar ~/steg_test default 0.2 Results.csv
(abhi@kali)~[/StegExpose]
$ cat Results.csv
File name,Above stego threshold?,Secret message size in bytes (ignore for clean files),Primary Sets,Chi Square,Sample Pairs,RS analysis,Fusion (mean)
(abhi@kali)~[/StegExpose]
$ java -jar StegExpose.jar ~/steg_test fast 0.2
(abhi@kali)~[/StegExpose]
$
```

8. Review the Results:

- For each image analyzed, review the scores and determine whether you need to investigate further.

Example:

bash

```
java -jar StegExpose.jar suspect_image.png
```

Output:

```
makefile
```

```
Analyzing suspect_image.png...
```

```
Result: 0.4
```

```
Steganography likely present
```

```

[abhi@kali]~$ java -version
openjdk version "21.0.7" 2025-04-15
OpenJDK Runtime Environment (build 21.0.7+6-Debian-1)
OpenJDK 64-Bit Server VM (build 21.0.7+6-Debian-1, mixed mode, sharing)

[abhi@kali]~$ git clone https://github.com/b3dk7/StegExpose.git
Cloning into 'StegExpose' ...
remote: Enumerating objects: 314, done.
remote: Total 314 (delta 0), reused 0 (delta 0), pack-reused 314 (from 1)
Receiving objects: 100% (314/314), 38.31 MiB | 9.77 MiB/s, done.
Resolving deltas: 100% (133/133), done.

[abhi@kali]~$ cd StegExpose
$ ls
ChiSquare.class      dissertation.pdf      ImageFileManager.class  PixelBenchmark.class    PrimarySets.java        roc.png      RunStegExpose.class  SamplePairs.java      testFolder
ChiSquare.java       Fuse.class           ImageFileManager.java   PixelBenchmark.java     README.md               RSAnalysis.class  RunStegExpose.java   'StegExpose analysis.ods'
commons-math3-3.1.1.jar Fuse.java            manifest.mf             PrimarySets.class       RSAnalysis.java         SamplePairs.class  SamplePairs.class    StegExpose.jar
[abhi@kali]~/StegExpose$ mkdir ~/steg_test

```

This output indicates that hidden data is likely present in the image suspect_image.png.

Rubrics

Criteria & Marks Assigned	Mark Allotted	Mark Awarded
1. GitHub Activity & Submission Regularity	3	
2. Application of Forensic Tools & Practical Execution	3	
3. Documentation & Reporting	2	
4. Engagement, Problem-Solving & Team Collaboration	2	
Total	10	

Result: Thus, StegExpose was successfully used to analyze images and detect potential hidden data through steganography detection scores.